The Journal of Open Source Software

FLIMKit: a Python toolkit for reconvolution fitting, lifetime-distribution fitting, and phasor analysis of FLIM data

Alexander Hunt [1¶], **Ava Russell** [1], **Layla Mathieson** [1], **Zhen Yuan Yeo** [2], **Matthieu Veremen** [1], **and Ahsan Akram** [1]

1 Centre for Inflammation Research, Institute for Regeneration and Repair, The University of Edinburgh, Edinburgh, United Kingdom **2** Department of Statistics and Data Science, National University of Singapore, Singapore ¶ Corresponding author

DOI: 10.xxxxxx/draft

Software

- Review
- Repository
- Archive

Editor:

Submitted: 03 September 2026
Published: unpublished

License
Authors of papers retain copyright and release the work under a Creative Commons Attribution 4.0 International License (CC BY 4.0).

DRAFT

Summary

Fluorescence lifetime imaging microscopy (FLIM) measures how long a fluorophore stays excited before emitting a photon. The lifetime reports on the local environment of the molecule (pH, oxygen tension, ion binding, proximity to an acceptor in Förster resonance energy transfer) and not on concentration or excitation intensity (Datta et al., 2020). The fluorophore can be a label or an endogenous species such as NAD(P)H and FAD, so FLIM applies label-free to tissue autofluorescence. That sensitivity underpins translational work: autofluorescence lifetime alone classifies human T cells by activation state with 97–99% accuracy (Walsh et al., 2021), discriminates lung cancer from adjacent tissue during fibre-based imaging (Fernandes et al., 2024), generates virtual haematoxylin-and-eosin histology from unstained tissue (Wang et al., 2024), and predicts EGFR mutation status in lung adenocarcinoma (Zang et al., 2026). Doing this at scale needs analysis software that can be scripted, reproduced and used with any vendor's instrument.

FLIMKit (v0.14.3) is a Python toolkit for FLIM data from time-correlated single-photon counting (TCSPC) instruments (Shcheslavskiy et al., 2025). PicoQuant `.ptu`, Becker & Hickl `.sdt`, Photonscore `.photons` and, experimentally, ISS files are read through one shared file abstraction. Three analysis routes are supported: reconvolution fitting of discrete multi-exponential models, fitting of continuous lifetime distributions, and calibrated phasor analysis (Digman et al., 2008; Ranjit et al., 2018). Per-pixel fitting is accelerated on Apple Silicon, NVIDIA CUDA and AMD ROCm hardware. FLIMKit runs as a desktop application, a guided terminal interface, command-line scripts or a Python API, and exports OME-TIFF, GeoJSON for QuPath, and CSV.

Statement of need

Laboratories running Leica FALCON/STELLARIS, Becker & Hickl SPC, Photonscore LINCam or ISS FastFLIM/Vista instruments mostly analyse their data with the vendor's software (LAS X, SPCImage, SymPhoTime). These packages are closed-source, graphical-only and tied to one file format, so batching, scripting and any analysis the vendor did not anticipate are hard, and a facility with more than one platform has no single reproducible pipeline. FLIMKit decodes every supported format into one `FLIMFile` abstraction, so the same fitting, phasor, stitching and export code runs whatever the instrument.

Open-source tools cover separate parts of the problem. FLIMfit (Warren et al., 2013) offers established reconvolution and global fitting but no phasor analysis, is built around PicoQuant

and OME-TIFF data, and has had no release since 2017. PhasorPy (Gohlke et al., n.d.), the `napari-flim-phasor-plotter` and `napari-phasors` plugins (Pannunzio et al., 2026; Zoccoler et al., 2024), FLIMPA (Kapsiani et al., 2025) and GSLab (Vallmitjana et al., 2025) implement the phasor method of Digman et al. (Digman et al., 2008), formalised as a fit-free protocol by Ranjit et al. (Ranjit et al., 2018). None performs reconvolution fitting with full instrument-response-function (IRF) deconvolution, and GSLab needs MATLAB. FLIMKit uses PhasorPy for the phasor method itself and adds calibration, interactive cursors, spatial filtering and ROI-based batch export on top. We designed FLIMKit for laboratories and imaging facilities that need vendor-independent analysis across these routes, for users who open, fit and export through the interface without writing code and for those building automated pipelines.

PicoQuant, Becker & Hickl and Photonscore support has been validated against real instrument files. ISS support is built from the vendor's format specifications, has not been checked against real ISS acquisitions, and is experimental.

State of the field

Table 1: Comparison of FLIM analysis software. FLIMKit's phasor analysis is built on PhasorPy.

Software	Access	Formats	Fitting + phasor	Scriptable
LAS X / SPCImage / SymPhoTime	Proprietary	Vendor-locked	Yes / partial	No
FLIMfit (Warren et al., 2013)	Open source	PicoQuant, OME-TIFF	Yes / no	Partial
PhasorPy (Gohlke et al., n.d.)	Open source	Many (readers)	No / yes	Yes
napari-flim-phasor-plotter / napari-phasors (Pannunzio et al., 2026; Zoccoler et al., 2024)	Open source	Several	No / yes	Via napari
FLIMPA (Kapsiani et al., 2025)	Open source	PTU, .sdt, .tif	No / yes (+ROI)	No
GSLab (Vallmitjana et al., 2025)	Open source	Several	No / yes	Via MATLAB
FLIMKit	Open source	PTU, .sdt, Photonscore, ISS (exp.)	Yes / yes	Yes

To our knowledge FLIMKit is the only open-source package that supports several instrument vendors behind one abstraction, performs both reconvolution fitting and phasor analysis, and is scriptable with GPU acceleration. The fitting tools (FLIMfit; TRI2/FLIMLib, an ImageJ library) are largely unmaintained, and the phasor tools do not fit decays. The phasor packages are more mature in their own domain, so FLIMKit builds on PhasorPy. The contribution is integration: one file abstraction, both analysis families in one interface, and three validated vendors plus an experimental fourth.

DRAFT

The Journal of Open Source Software

Software design

The central design decision is the `FLIMFile` abstraction. Where mature decoders exist the readers wrap `ptufile` and `sdtfile` (both by Gohlke) for PicoQuant and Becker & Hickl data and `photonsfile` (Hunt & Akram, 2026) for Photonscore, and reduce their output to a common `(Y, X, H)` decay-cube-and-metadata contract. No instrument-specific field passes further into the pipeline, which constrains new readers but gives one tested code path instead of one per format. Photonscore files record each photon with its own (x, y) position and micro-time, so the image is formed by binning positions and the decay by histogramming micro-times. `photonsfile` was written for FLIMKit, released as a standalone package, and validated bit-exact against the vendor SDK.

Per-pixel fitting is the dominant computational cost: a 1024×1024 acquisition requires of the order of 10^6 independent non-linear fits. No single GPU interface spans Apple Silicon, NVIDIA and AMD hardware, so a backend abstraction (MLX, PyTorch MPS/CUDA/ROCm, numba, or a CPU fallback) lets one `fit_per_pixel()` call run on whatever is available. Photon counts per pixel are low and Poisson distributed, so when the lifetimes are fixed the amplitudes are estimated by maximum likelihood under a Poisson model with a fitted constant offset. The global ('summed') fit stays on the CPU, where it is not a bottleneck, and reports standard errors for every parameter. For three-exponential fits the errors from the curvature of the objective can be several times too small, so we also resample the decay under Poisson noise and refit. On one synthetic three-exponential decay the relative error on τ_2 was 5.8% from curvature and 24% from the bootstrap, against 51% observed over repeated noise realisations.

Two lifetime models cover two situations: a discrete multi-exponential model for a small fixed number of species, and a continuous Gaussian/Lorentzian $\alpha(\tau)$ distribution following Lakowicz (Lakowicz, 2006) for heterogeneous populations, such as autofluorescence, that a fixed-species model would misrepresent. A time-varying background correction follows Görlitz et al. (Görlitz et al., 2017).

Multi-tile acquisitions are stitched before fitting. Tile positions come from the stage metadata and are refined by cross-correlation between neighbours, following Preibisch et al. (Preibisch et al., 2009), so fitting and phasor analysis see one continuous field of view. For batch ROI workflows, cell masks come from the summed intensity image using Cellpose-SAM (Pachitariu et al., 2025), and per-cell lifetime and phasor statistics are exported without a separate segmentation step (Figure 1).

DRAFT

The Journal of Open Source Software

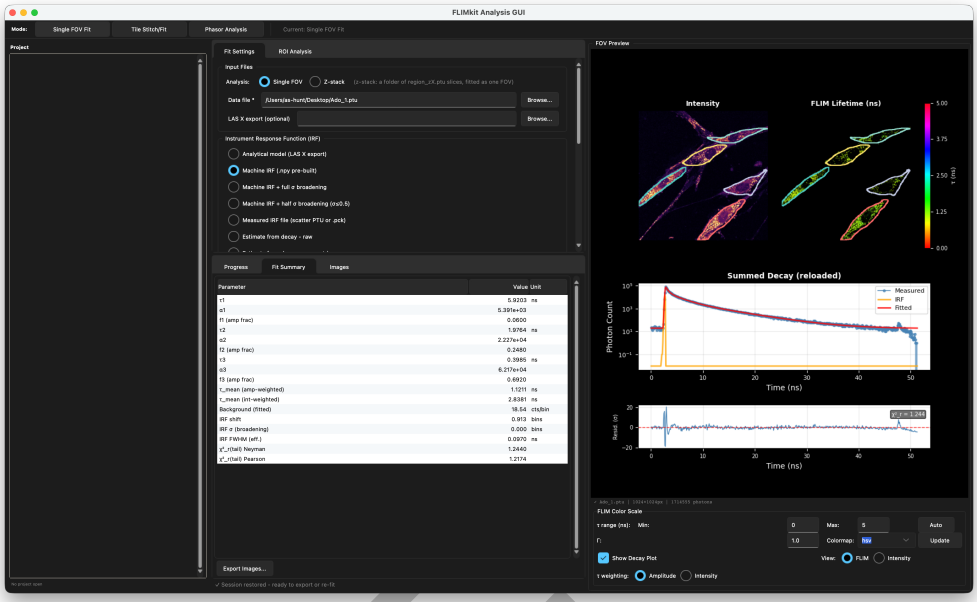


Figure 1: The FLIMKit desktop interface in Single FOV Fit mode. Five freehand regions of interest outline individual cells on the intensity image and the per-pixel lifetime map. Below are the summed decay with its reconvolution fit and IRF, and the weighted residuals. The Fit Summary lists the recovered tri-exponential parameters, both mean lifetimes, the fitted background and IRF shift, and reduced χ^2. Per-ROI statistics are in the Regions table. Data are a PicoQuant `.ptu` acquisition (1024×1024 pixels, 1.7×10^6 photons).

Capability that a laboratory needs but a general toolkit should not carry goes in through a plugin registry. A plugin is a Python module that registers against one of seven hooks (analysis tools, ROI panel buttons, startup actions, file formats, format sniffers, phasor filters and per-pixel fit backends) and declares the registry API version it was written for; a mismatch is refused, and a plugin that fails on import cannot stop the application starting. FLIMKit's own tools use the same registry, and a fixture plugin is loaded by the test suite on every run, so a change that breaks the contract fails in continuous integration. QuPath and Fiji bridges, a time-resolved anisotropy package and an optional C and CUDA fit accelerator ship as separate packages on this basis.

The four entry points call the same fitting and phasor functions, so an interactive analysis can be reproduced in a script unchanged. The test suite has 74 test modules covering CPU and GPU numerical parity, ground-truth lifetime recovery and format decoding. Compiled desktop applications for macOS, Linux and Windows are built on each tagged release, and Docker images are maintained in a separate repository. Installation, entry points and deployment are in the documentation.

Research impact statement

FLIMKit is the routine analysis route for FLIM data at the Centre for Inflammation Research, University of Edinburgh, where we have used it on PicoQuant and Leica FALCON/STELLARIS acquisitions, including work in preparation.

Format support was developed with the instrument vendors. Becker & Hickl support was built and checked against sample data and format documentation from the vendor, Photonscore support against a sample `.photons` file and SDK documentation from Photonscore, and ISS format specifications were provided by ISS after a discussion on the CONFOCALMICROSCOPY

DRAFT

listserv. FLIMKit is listed among phasor-analysis tools in the PhasorPy documentation, and has been applied to independently acquired datasets at the CRUK Scotland Institute and the National University of Singapore. This use is early and exploratory and has not been validated as a deployment. Wider external adoption remains a priority.

AI usage disclosure

FLIMKit is designed, developed and maintained by the authors. Anthropic's Claude was used as an assistant for parts of the implementation. All scientific design, fitting and phasor methods, validation and the overall architecture are the authors' own work. All AI-generated content was reviewed, edited and tested by the authors on both real and synthetic FLIM data, and the authors remain responsible for the implementation and its scientific validity.

Contributions by Z.Y.Y. were developed with assistance from OpenAI's GPT-5.6 Sol, run through Hermes Agent by Nous Research. Z.Y.Y. directed and reviewed that work, and the authors each independently reviewed and tested the AI-assisted contributions for scientific validity and correctness.

Acknowledgements

We thank Christoph Gohlke for PhasorPy, `ptufile`, `lfdfiles` and `sdtfile`; Dr. Jens Balke and Enzo Marscheck (Becker & Hickl) for `.sdt` documentation and sample data; Photonscore GmbH for the LINCam SDK, a sample `.photons` file, and for open-sourcing the D7 format; and Jeff Liao (ISS) for ISS format specifications.

References

Datta, R., Heaster, T. M., Sharick, J. T., Gillette, A. A., & Skala, M. C. (2020). Fluorescence lifetime imaging microscopy: Fundamentals and advances in instrumentation, analysis, and applications. *Journal of Biomedical Optics*, *25*(7), 071203. https://doi.org/10.1117/1.JBO.25.7.071203

Digman, M. A., Caiolfa, V. R., Zamai, M., & Gratton, E. (2008). The phasor approach to fluorescence lifetime imaging analysis. *Biophysical Journal*, *94*(2), L14–L16. https://doi.org/10.1529/biophysj.107.120154

Fernandes, S., Williams, E., Finlayson, N., Stewart, H., Dhaliwal, C., Dorward, D. A., Wallace, W. A., Akram, A. R., Stone, J., Dhaliwal, K., & Williams, G. O. S. (2024). Fibre-based fluorescence-lifetime imaging microscopy: A real-time biopsy guidance tool for suspected lung cancer. *Translational Lung Cancer Research*, *13*(2), 355–361. https://doi.org/10.21037/tlcr-23-638

Gohlke, C., Pannunzio, B., Schuty, B., & Blanco, R. (n.d.). *PhasorPy: An open-source python library for phasor analysis of fluorescence lifetime and hyperspectral images*. Zenodo. https://doi.org/10.5281/zenodo.13862586

Görlitz, F., Kelly, D. J., Warren, S. C., Alibhai, D., West, L., Kumar, S., Alexandrov, Y., Munro, I., Garcia, E., McGinty, J., Talbot, C., Serwa, R. A., Thinon, E., Paola, V. da, Murray, E. J., Stuhmeier, F., Neil, M. A. A., Tate, E. W., Dunsby, C., & French, P. M. W. (2017). Open source high content analysis utilizing automated fluorescence lifetime imaging microscopy. *Journal of Visualized Experiments*, *119*, e55119. https://doi.org/10.3791/55119

Hunt, A., & Akram, A. (2026). *Photonsfile: A pure-python reader for photonscore LINCam .photons (D7) files*. Zenodo. https://doi.org/10.5281/zenodo.21360199

Kapsiani, S., Läubli, N. F., Ward, E. N., Shehata, M., Kaminski, C. F., & Kaminski Schierle,

DRAFT

G. S. (2025). FLIMPA: A versatile software for fluorescence lifetime imaging microscopy phasor analysis. *Analytical Chemistry*. https://doi.org/10.1021/acs.analchem.5c00495

Lakowicz, J. R. (2006). *Principles of fluorescence spectroscopy* (3rd ed.). Springer. https://doi.org/10.1007/978-0-387-46312-4

Pachitariu, M., Rariden, M., & Stringer, C. (2025). Cellpose-SAM: Superhuman generalization for cellular segmentation. *bioRxiv*. https://doi.org/10.1101/2025.04.28.651001

Pannunzio, B., Zoccoler, M., Rivero, J., & Schuty, B. (2026). *Napari-phasors* (Version 0.7.0). Zenodo. https://doi.org/10.5281/zenodo.14647626

Preibisch, S., Saalfeld, S., & Tomancak, P. (2009). Globally optimal stitching of tiled 3D microscopic image acquisitions. *Bioinformatics*, *25*(11), 1463–1465. https://doi.org/10.1093/bioinformatics/btp184

Ranjit, S., Malacrida, L., Jameson, D. M., & Gratton, E. (2018). Fit-free analysis of fluorescence lifetime imaging data using the phasor approach. *Nature Protocols*, *13*(9), 1979–2004. https://doi.org/10.1038/s41596-018-0026-5

Shcheslavskiy, V. I., Shirmanova, M. V., Yashin, K. S., Rück, A. C., Skala, M. C., & Becker, W. (2025). Fluorescence lifetime imaging techniques—a review on principles, applications and clinical relevance. *Journal of Biophotonics*, *18*(12), e202400450. https://doi.org/10.1002/jbio.202400450

Vallmitjana, A., Torrado, B., Durkin, A. F., Dvornikov, A., Rajil, N., Ranjit, S., & Balu, M. (2025). GSLab: Open-source platform for advanced phasor analysis in fluorescence microscopy. *Bioinformatics*, *41*(4), btaf162. https://doi.org/10.1093/bioinformatics/btaf162

Walsh, A. J., Mueller, K. P., Tweed, K., Jones, I., Walsh, C. M., Piscopo, N. J., Niemi, N. M., Pagliarini, D. J., Saha, K., & Skala, M. C. (2021). Classification of t-cell activation via autofluorescence lifetime imaging. *Nature Biomedical Engineering*, *5*(1), 77–88. https://doi.org/10.1038/s41551-020-0592-z

Wang, Q., Akram, A. R., Dorward, D. A., Talas, S., Monks, B., Thum, C., Hopgood, J. R., Javidi, M., & Vallejo, M. (2024). Deep learning-based virtual H&E staining from label-free autofluorescence lifetime images. *Npj Imaging*, *2*(1), 17. https://doi.org/10.1038/s44303-024-00021-7

Warren, S. C., Margineanu, A., Alibhai, D., Kelly, D. J., Talbot, C., Alexandrov, Y., Munro, I., Katan, M., Dunsby, C., & French, P. M. W. (2013). Rapid global fitting of large fluorescence lifetime imaging microscopy datasets. *PLOS ONE*, *8*(8), e70687. https://doi.org/10.1371/journal.pone.0070687

Zang, Z., Dorward, D. A., Ihuoma, S., Akram, A. R., & Wang, Q. (2026). Label-free prediction of EGFR mutation status using fluorescence lifetime imaging and deep learning in lung adenocarcinoma. *Cancer Research*. https://doi.org/10.1158/0008-5472.CAN-25-5589

Zoccoler, M., Iarovenko, S., & Wetzker, C. (2024). *Napari-flim-phasor-plotter* (Version 0.1.0). Zenodo. https://doi.org/10.5281/zenodo.12620956

DRAFT